COCOME – THE COMMON COMPONENT MODELLING

Example

Sebastian Herold és társai

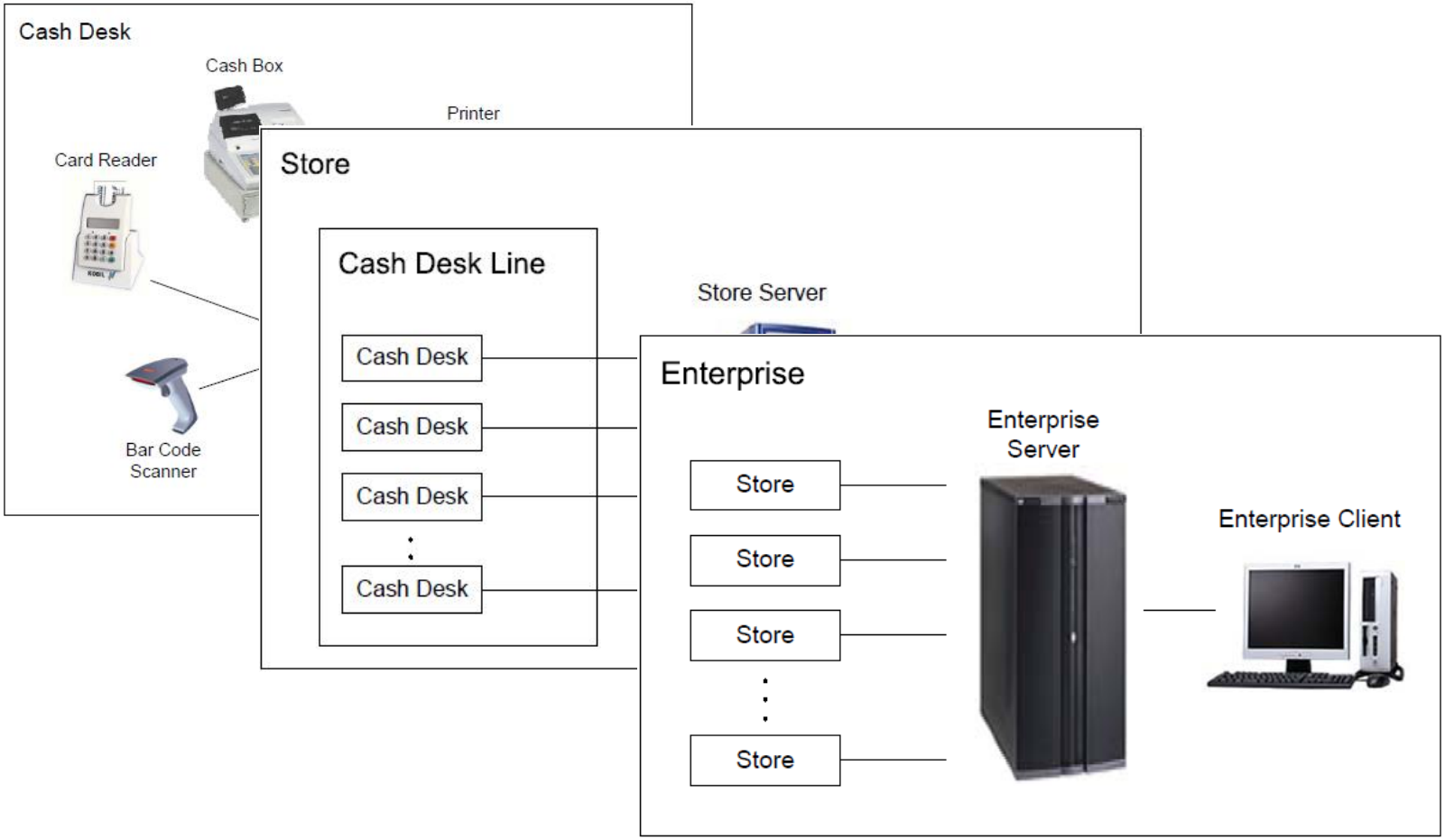
Germany: TU Clausthal, University Karlsruhe, Technishe Universitat München;

Italy: Politechnico di Milano

Egy kereskedelmi rendszer modellezése

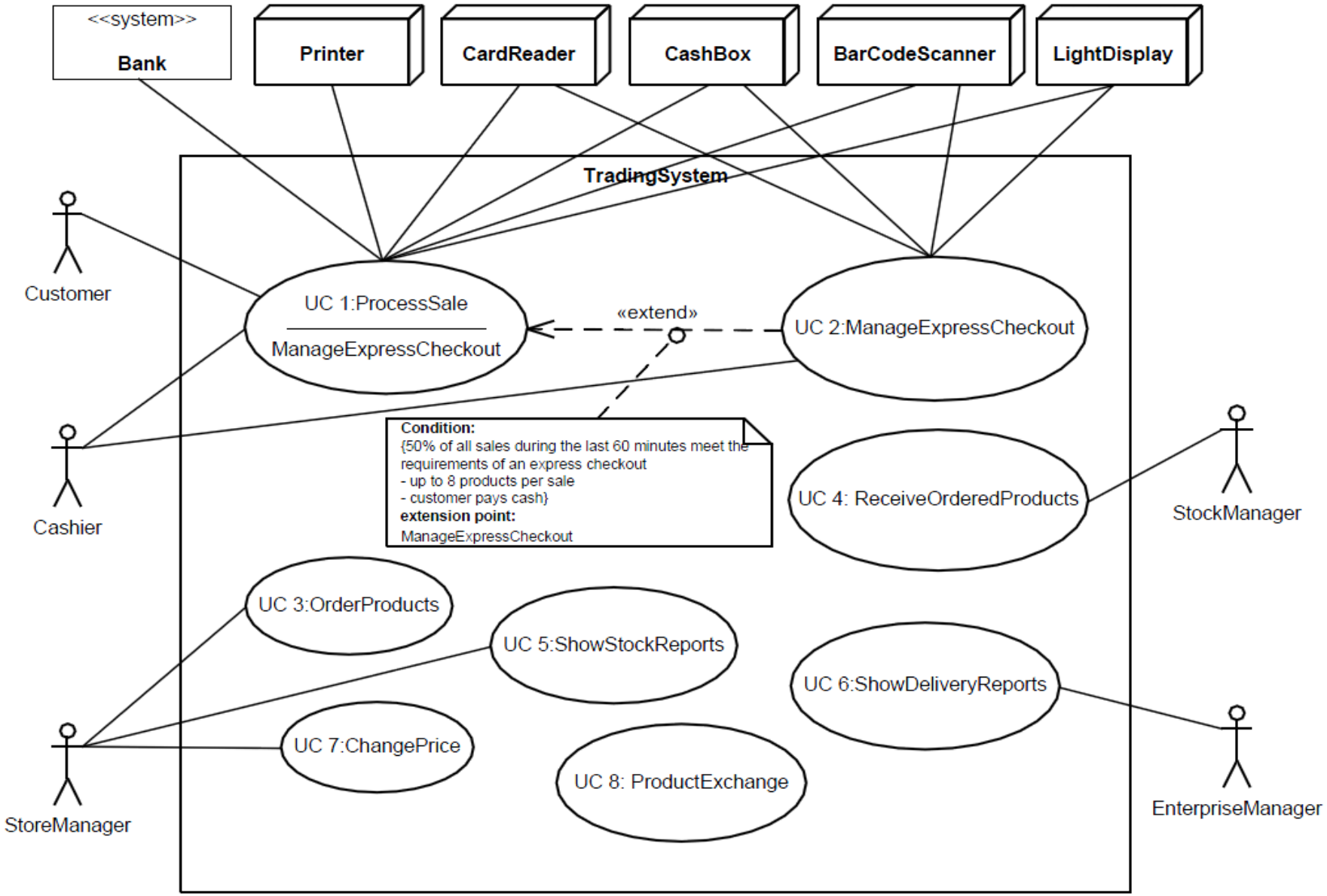
- Adott egy szupermarket kereskedelmi lánc, amely egy vállaltot alkot
  - A vállalathoz több áruház tartozik
  - Egy áruházában a pénztárnál fizethet az ügyfél a kiválasztott áruért
  - Egy áruházban több pénztár működik
  - A pénztárnál az ügyfél készpénzzel vagy bankkártyával fizethet.
  - A pénztáros vonalkód leolvasóval dolgozik
  - A pénztárgépek az áruház szerveréhez csatlakoznak és ezen keresztül a bankhoz

Áttekintés


Cash Desk
Cash Box
Printer
Card Reader
Bar Code Scanner
Store
Cash Desk Line
Cash Desk
Cash Desk
Cash Desk
Cash Desk
Store Server
Enterprise
Enterprise Server
Store
Store
Store
Store
Enterprise Client

Funkcionális követelmények és használati eset analízis

Functional Requirements and Use Case Analysis


<<system>>
Bank
Printer
CardReader
CashBox
BarCodeScanner
LightDisplay
TradingSystem
Customer
UC 1:ProcessSale
ManageExpressCheckout
«extend»
UC 2:ManageExpressCheckout
Cashier
Condition:
{50% of all sales during the last 60 minutes meet the requirements of an express checkout
- up to 8 products per sale
- customer pays cash}
extension point:
ManageExpressCheckout
UC 4: ReceiveOrderedProducts
StockManager
UC 3:OrderProducts
UC 5:ShowStockReports
UC 6:ShowDeliveryReports
UC 7:ChangePrice
UC 8: ProductExchange
StoreManager
EnterpriseManager

Használati eset sablon tartalma

- A használati eset sablon (use case template):
  - Rövid leírás (Brief Description)
  - Résztvevők (Involved Actors)
  - Előfeltétel (Precondition)
  - Kiváltó ok (Trigger)
  - Utófeltétel (Postcondition)
  - Általános eljárás (Standard Process)
  - Alternatív, kivételes eljárás (Alternative or Exceptional Processes)

Használati eset: az árusítás folyamata (UC 1)

Rövid leírás:

Egy ügyfél árúinak összegzése majd fizetés bankkártyával vagy készpénzzel

Résztvevők:

Ügyfél, Pénztáros, Bank, Nyomtató, Kártyaolvasó, Pénztárgép, Vonalkód olvasó, Kijelző

Előfeltétel:

A Pénztár, és a Pénztáros készen áll egy új árusítás végrehajtására

Kiváltó ok:

Egy Ügyfél érkezik a pénztárhoz és a kiválasztott áruért fizetni akar

Utófeltétel:

Az Ügyfél fizetett, megkapta a számlát, és az árusítást regisztrálták a Leltárban (az árukészletben)

Általános eljárás (UC 1)

- 1. The Customer arrives at the Cash Desk with goods to purchase. [arr1]
- 2. The Cashier starts a new sale by pressing the button Start New Sale at theCash Box. [t12-1]
- 3. The Cashier enters the item identifier. This can be done manually by using the keyboard of the Cash Box [p13-1, t13-1] or by using the Bar Code Scanner. [p13-2, t13-2]
- 4. Using the item identifier the System presents the corresponding product description, price, and running total. [t14-1]

  The steps 3-4 are repeated until all items are registered.

Általános eljárás-2 (UC 1)

- 5. Denoting the end of entering items the Cashier presses the button Sale Finished
  at the Cash Box.
  - (a) To initiate cash payment the Cashier presses the button Cash Payment
    at the Cash Box.
    - i. The Customer hands over the money for payment.
    - ii. The Cashier enters the received cash using the Cash Box and confirms
      this by pressing Enter.
    - iii. The Cash Box opens.
    - iv. The received money and the change amount are displayed and the Cashier hands over the change.
    - v. The Cashier closes the Cash Box.

Általános eljárás-3 (UC 1)

- (b) In order to initiate card payment the Cashier presses the buttom *Card Payment* at the Cash Box
  - i.The Cashier receives the credit card from the Customer and pulls it through the Card Reader.
  - ii. The customer enters his PIN using the keyboard of the card reader and waits for validation. *The step 5.b.ii is repeated until a successful validation or the Cashier presses the buttom for cash payment.*

Általános eljárás-4 (UC 1)

- 6. Completed sales are logged by the Trading System and sale information are sent to the Inventory in order to update the stock.
- 7. The Printer writes the receipt and the Cashier hands it out to the Customer.
- 8. The customer leaves the Cash Desk with receipt and goods.

Alternatív vagy kivételes eljárás (UC 1)

- – In step 3: Invalid item identifier if the system cannot find it in the Inventory.
  - 1. The System signals error and rejects this entry.
  - 2. The Cashier can respond to the error as follows:
    - (a) It exists a human-readable item identifier:
      - i. The Cashier manually enters the item identifier.
      - ii. The System displays the description and price.
    - (b) Otherwise the product item is rejected.

Alternatív vagy kivételes eljárás 2 (UC 1)

- – *In step 5.b: Card validation fails.*
  - 1. The Cashier and the Customer try again and again.
  - 2. Otherwise the Cashier requires the Customer to pay cash.

- – *In step 6: Inventory not available*.
  The System caches each sale and writes them into the Inventory as soon as it is available again.

További használati esetek

- Az UC 1 használati esethez hasonlóan leírhatók a további használati esetek is
  - UC 2 Manage Express Checkout
    - Ha bizonyos feltételek teljesülnek a pénztár automatikusan expressz módba kapcsol.
    - Ebben az esetben maximum 8 tétel vásárolható és csak készpénzzel lehet fizetni.
    - …
    - Megjegyzés: figyeljük meg az *Extending Use Cases* és az *Extension point* használatát (az UC 1 használati esetet terjesztjük ki az UC 2 használati eset szolgáltatásaival).
  - UC 3 Order Products
  - UC 4 Receive Ordered Products

UC 5 Show Stock Reports

- **Rövid leírás:** A Store Manager az adott áruházban lévő árukról statisztikai jelentést készíttethet
- **Résztvevők**: Store Manager
- **Előfeltétel:** A jelentést készítő GUI és a Store Kliens működik
- **Kiváltó ok:** A Store Manager látni akarja az áruház statisztikai adatait
- **Utófeltétel:** A jelentés elkészült és megjelenítésre került
- **Általános eljárás:**
  - 1. A Store Manager beüti az áruház azonosítóját és megnyomja a *Jelentés Készítése* gombot.
  - 2. Az áruházban található teljes készlet megjelenítésre kerül.

További használati esetek 2

- UC 6 Show Delivery Reports
  - The Trading System provides the opportunity to calculate the mean times a delivery from each supplier to a considered enterprise takes.
- UC 7 Change Price
  - The Trading System provides the opportunity to change the sales price for a product.
- UC 8 Product Exchange (on low stock) Among Stores
  - If a store runs out of a certain product, it is possible to start a query whether those products are available at other Stores of the Enterprise. After a successful query the critical product can be shipped from one to other Stores. But … (további izgalmas feltételek megfogalmazhatók).

Extra-funkcionális tulajdonságok

Extra-Functional Properties

Extra-funkcionális tulajdonságok

- A kereskedelmi rendszer extra-funkcionális tulajdonságainak leírását táblázatos formában adják meg a szerzők, megadva azokat a helyeket a konkrét használati eset sablonokban, ahol az adott tulajdonság jelentőséggel bír.
- Ezek a tulajdonságok vezérfonalként szolgálnak amikor a különböző minőségi szempontok szerint vizsgáljuk a rendszerünket.
- Néhány tulajdonság:
  - Időzítés (timing),
  - Megbízhatóság (reliability),
  - Használati profil (usage profile related information).
- A következő ábrán bemutatunk egy ilyen táblázatnak egy szeletét az UC 1 használati esetre vonatkozóan. A többi használati esetre újabb és újabb funkcionális tulajdonságok vezethetők be.
  - A táblázatok leírására az OMG UML Profile for Schedulability, Performance, and Time anyagában található nyelvet használták a szerzők.

Egy ilyen táblázat egy részlete

- UC 1 Árusítás folyamata (Process sale)
  - arr1: Customer arrival rate per store: *adott érték*
  - t12-1: Time for pressing buttom „Start New Sale”: *adott érték*
  - t13-1: Time for scanning an item: *histogrammal adott*
  - t13-2: Time for manual entry: *adott érték*
  - t14-1: Time for showing the product description, price, and running total: *adott érték*
  - p13-1: Probability of using the bar code scanner per item: *0.99*
  - pt13-2: Probability of manual entry per item: *0.01*
  - …
  - Az UC 1 használati esethez a teljes táblázat 37 tételt tartalmaz.
- A többi használati esethez a megfelelő táblázatok hasonló módon megkonstruálhatók.

A kereskedelmi rendszer architektúrája

The architecture of the *Trading System*

A kereskedelmi rendszer struktúrája

- A következő ábra a *TradingSystem* szuperkomponenst mutatja be, amely két további komponensből épül fel:
  - Inventory
    - komponens reprezentálja az információs rendszert,
  - CashDeskLine
    - reprezentálja a beágyazott rendszert.

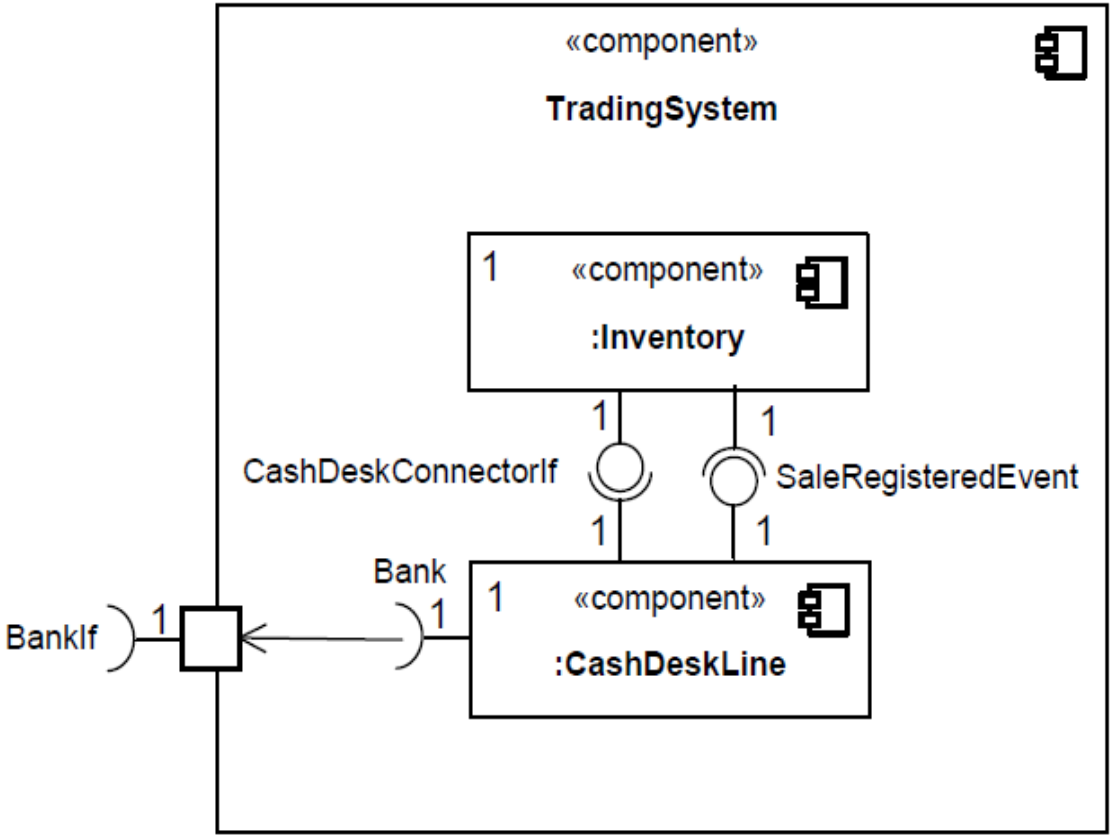
TradingSystem

- **Inventory**

  Információs rendszer – régeteges felépítéssel

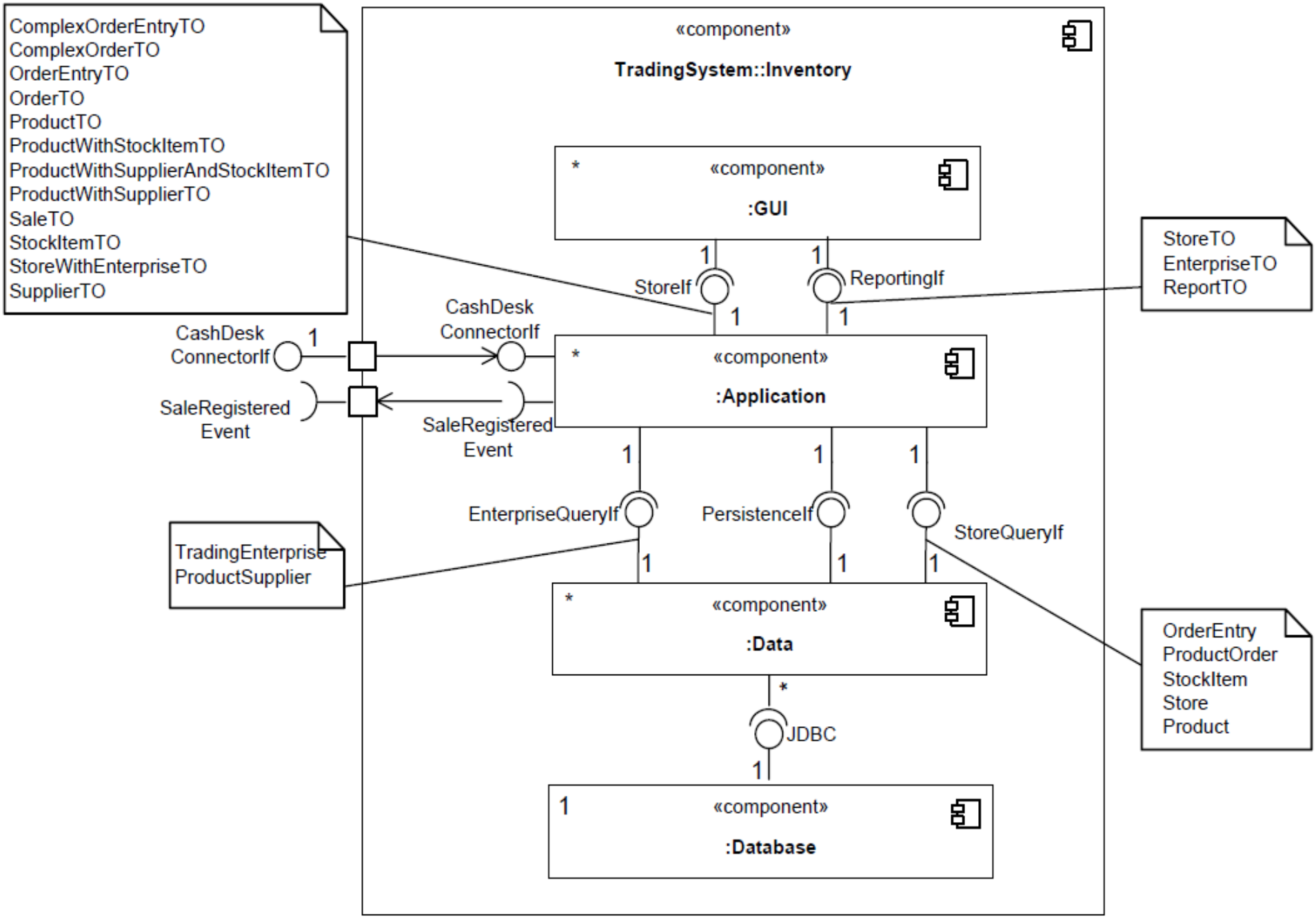
- **Pénztársor (Cash Desk Line)**

  Beágyazott rendszerek egy busz architektúrán



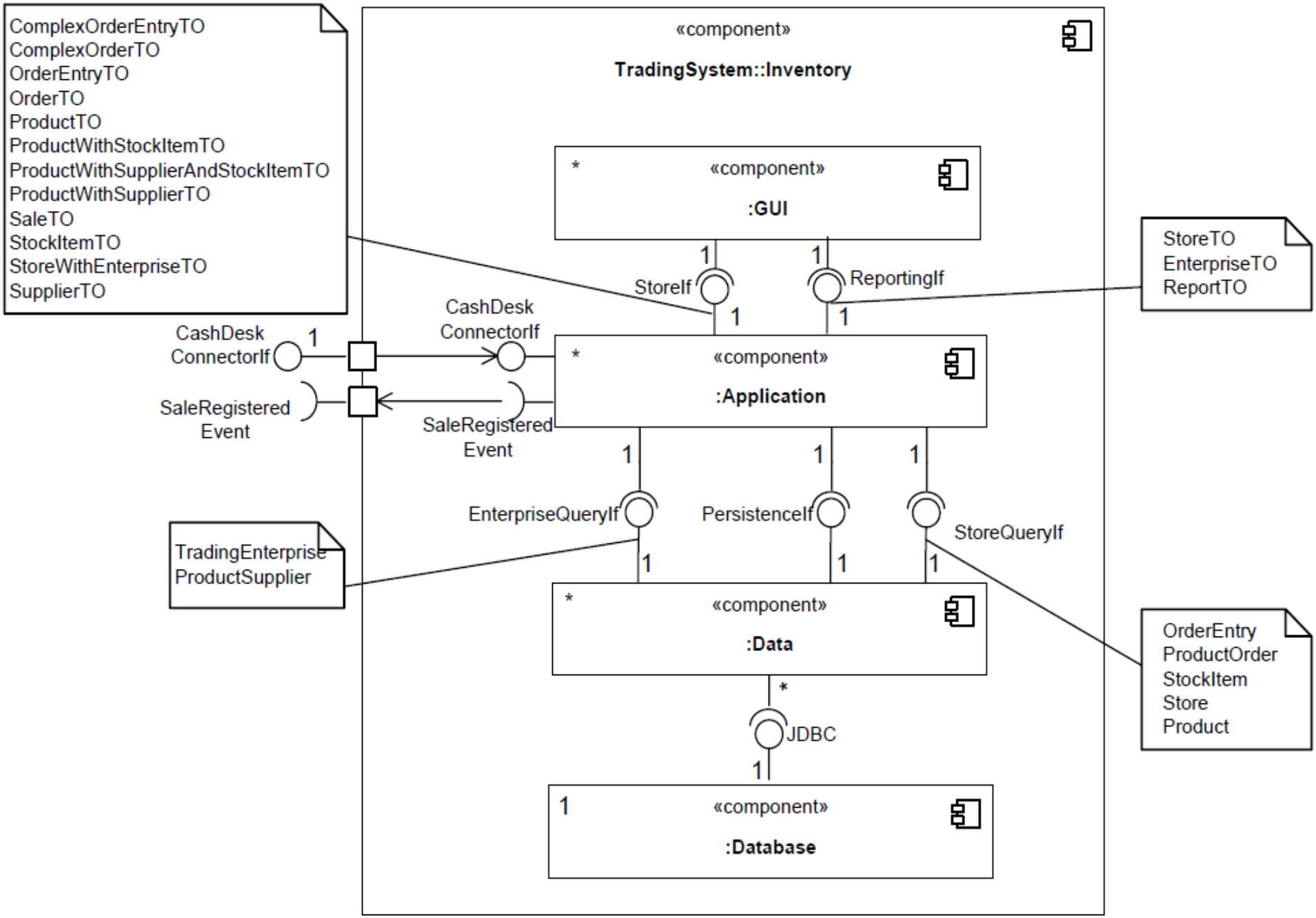
A *TradingSystem* szuperkomponens

- A *TradingSystem a fenti* két komponens egy-egy példányából épül fel, ezt jelzi a komponensek bal felső sarában található 1 számjegy.
- A két komponens közötti kommunikációt a következő két interfész kezeli
  - *CashDeskConnectorIf*
    - definiál egy metódust, amellyel az árucikkről kaphatunk információt az árucikk kódja alapján: leírás, ár;
  - *SaleRegisteredEvent*
    - Az esemény interfész ebben az esetben egy új eladás indulását regisztrálja egy aszinkron esemény csatornán.
- A *CashDeskLine* komponens a bankkal van kapcsolatban a *BankIf* interfészen keresztül, amelynek feladata a kártyás fizetési mód kezelése. A komponens egy kapun keresztül tartja a kapcsolatot a bankkal.


ComplexOrderEntryTO
ComplexOrderTO
OrderEntryTO
OrderTO
ProductTO
ProductWithStockItemTO
ProductWithSupplierAndStockItemTO
ProductWithSupplierTO
SaleTO
StockItemTO
StoreWithEnterpriseTO
SupplierTO
«component»
TradingSystem::Inventory
«component»
:GUI
StoreIf
ReportingIf
StoreTO
EnterpriseTO
ReportTO
CashDesk
ConnectorIf
CashDesk
ConnectorIf
SaleRegistered
Event
SaleRegistered
Event
«component»
:Application
EnterpriseQueryIf
PersistenceIf
StoreQueryIf
TradingEnterprise
ProductSupplier
«component»
:Data
JDBC
OrderEntry
ProductOrder
StockItem
Store
Product
«component»
:Database

Az Inventory komponens strukturális nézete

- Az előző ábra az Inventory komponens belső struktúráját mutatja be, amely a kereskedelmi rendszer információs rendszerét reprezentálja.
- Az architektúra a következő rétegekből épül fel:
  - Gui,
  - Application,
  - Data,
  - Database.
- Az Inventori komponens mindenegyes példányához az adatbázis (Database) egyetlen példánya létezik
- A Data réteg egy klasszikus háromrétegű architektúra, amelyet később részletesen bemutatunk.
  - A Database és Data rétegek között a kommunikációt a JDBC menedzseli.
  - A Data komponens három interfészen keresztül kapcsolódik az Application réteghez (*EnterpriseQuiryIf, StoreQuiryIf, PersistenceIf)*


ComplexOrderEntryTO
ComplexOrderTO
OrderEntryTO
OrderTO
ProductTO
ProductWithStockItemTO
ProductWithSupplierAndStockItemTO
ProductWithSupplierTO
SaleTO
StockItemTO
StoreWithEnterpriseTO
SupplierTO
«component»
TradingSystem::Inventory
«component»
:GUI
StoreIf
ReportingIf
StoreTO
EnterpriseTO
ReportTO
CashDesk
ConnectorIf
CashDesk
ConnectorIf
SaleRegistered
Event
SaleRegistered
Event
«component»
:Application
EnterpriseQueryIf
PersistenceIf
StoreQueryIf
TradingEnterprise
ProductSupplier
«component»
:Data
JDBC
OrderEntry
ProductOrder
StockItem
Store
Product
«component»
:Database

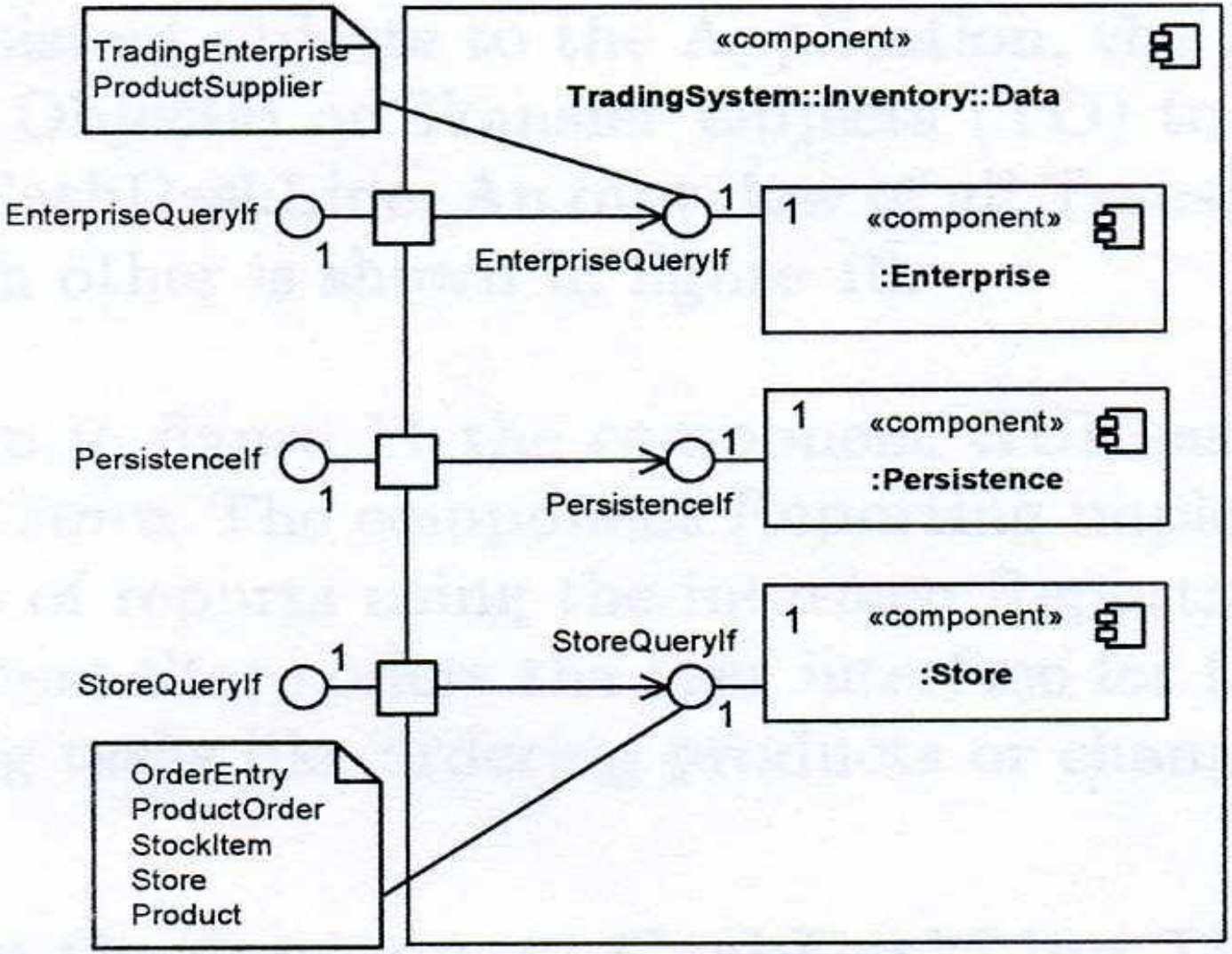
- Az *EnterpriseQuiryIf interfész lekérdezéseket tartalmaz a vállalat működésére nézve.*
- *StoreQuiryIf az áruházak számára metódusokat definiál*
  - *az eladási árak megváltoztatására,*
  - *az Inventory menedzselésére,*
  - *stb.*
- *PersistenceIf)*
  - *perzisztens összefüggések lekérdezésére nyújt egy metódust.*

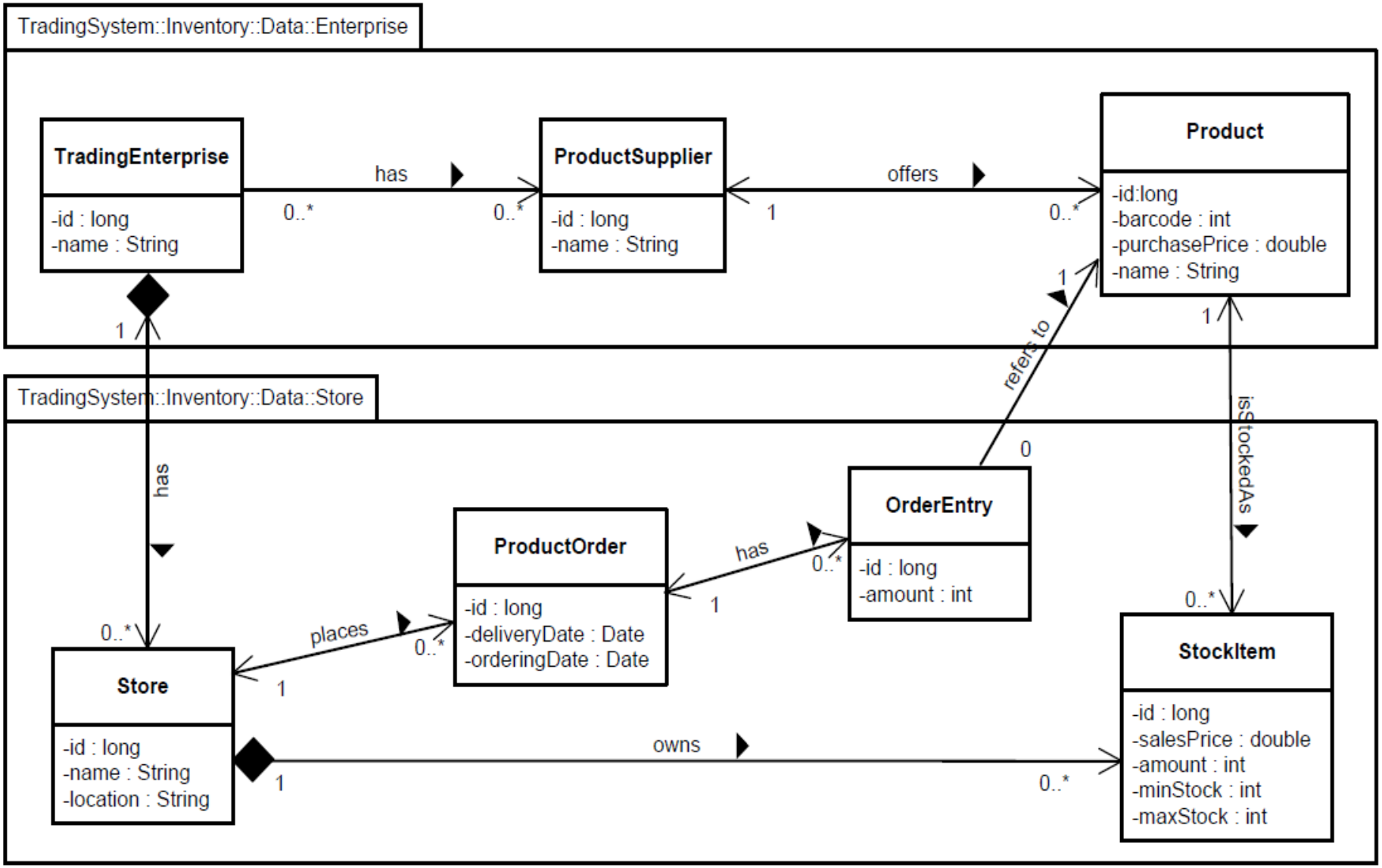
- Az Application komponens az alkalmazási logikát tartalmazza.
  - A Data komponens interfészeit használja, hogy lekérdezéseket és változtatásokat küldjön a Database számára.
  - A *StoreIf* és *ReportingIf* interfészeket definiálja a GUI komponens számára, amelyeken keresztül az alkalmazási réteg eljuttatja az adatbázishoz küldött lekérdezések eredményét a GUI rétegnek.
- Az alkalmazási és GUI rétegek (komponensek) között nem referenciákat adunk át, hanem az adatok közvetítésére, átadására *az úgynevezett Transfer Objects (TO) objektumok szolgálnak.*

Data réteg

- A következő ábra az adatréteget mutatja be, amely három alkomponenst tartalmaz:
  - *Enterprise,*
  - *Persistence,*
  - *Store.*
- Ezen komponensek feladata a megfelelő *EnterpriseQuiryIf, StoreQuiryIf, PersistenceIf* interfészek megvalósítása.
- A kereskedelmi rendszer adatmodellje részletesebb áttekintést ad az attribútumokkal és navigációs utakkal kiegészített adatmodellről.

A Inventory komponens adatrétegének belső struktúráia

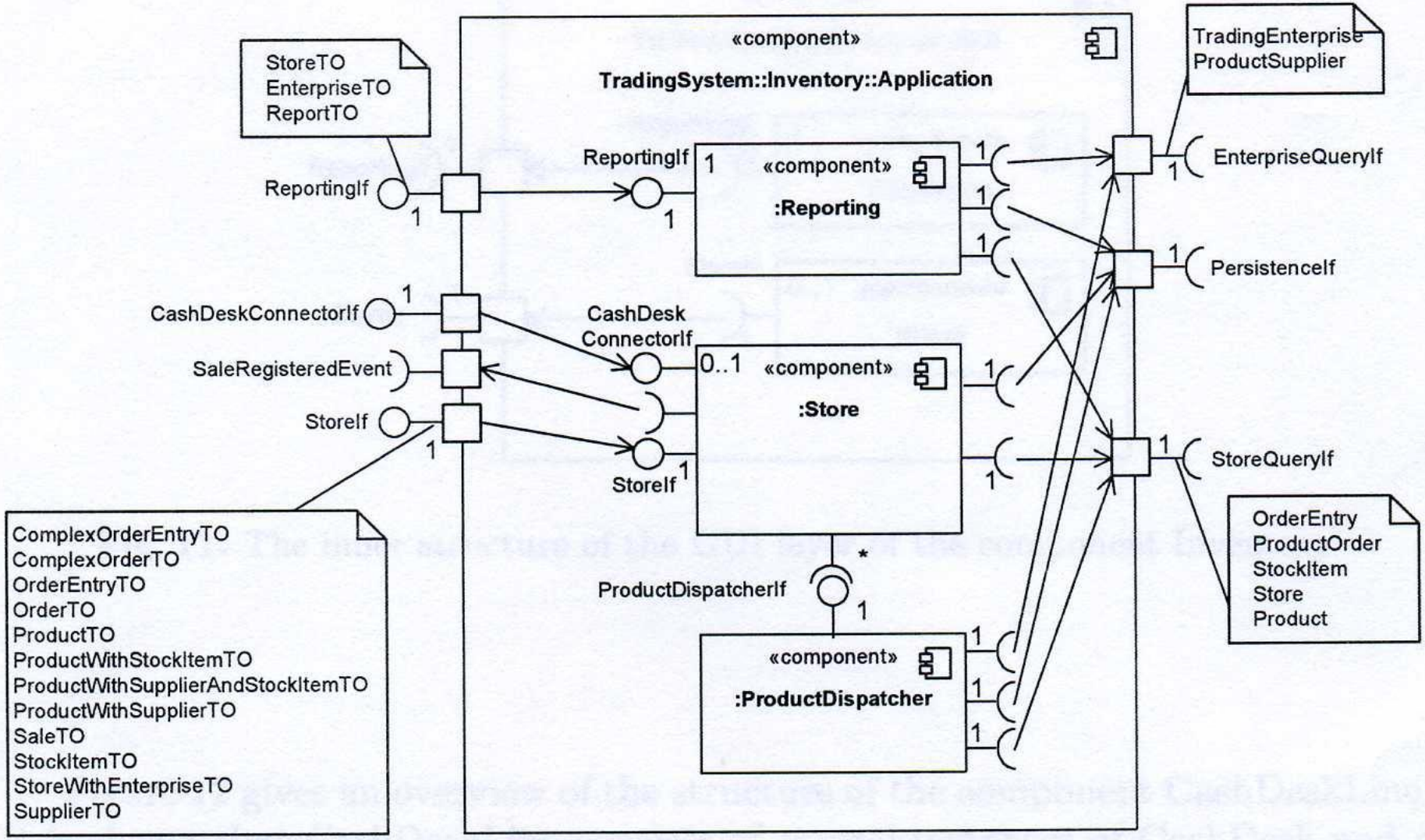



TradingSystem::Inventory::Data::Enterprise
TradingEnterprise
-id : long
-name : String
has
0..*
0..*
ProductSupplier
-id : long
-name : String
offers
1
0..*
Product
-id:long
-barcode : int
-purchasePrice : double
-name : String
1
1
refers to
0
1
TradingSystem::Inventory::Data::Store
has
isStockedAs
OrderEntry
-id : long
-amount : int
ProductOrder
-id : long
-deliveryDate : Date
-orderingDate : Date
has
0..*
1
0..*
places
0..*
1
Store
-id : long
-name : String
-location : String
owns
1
0..*
StockItem
-id : long
-salesPrice : double
-amount : int
-minStock : int
-maxStock : int

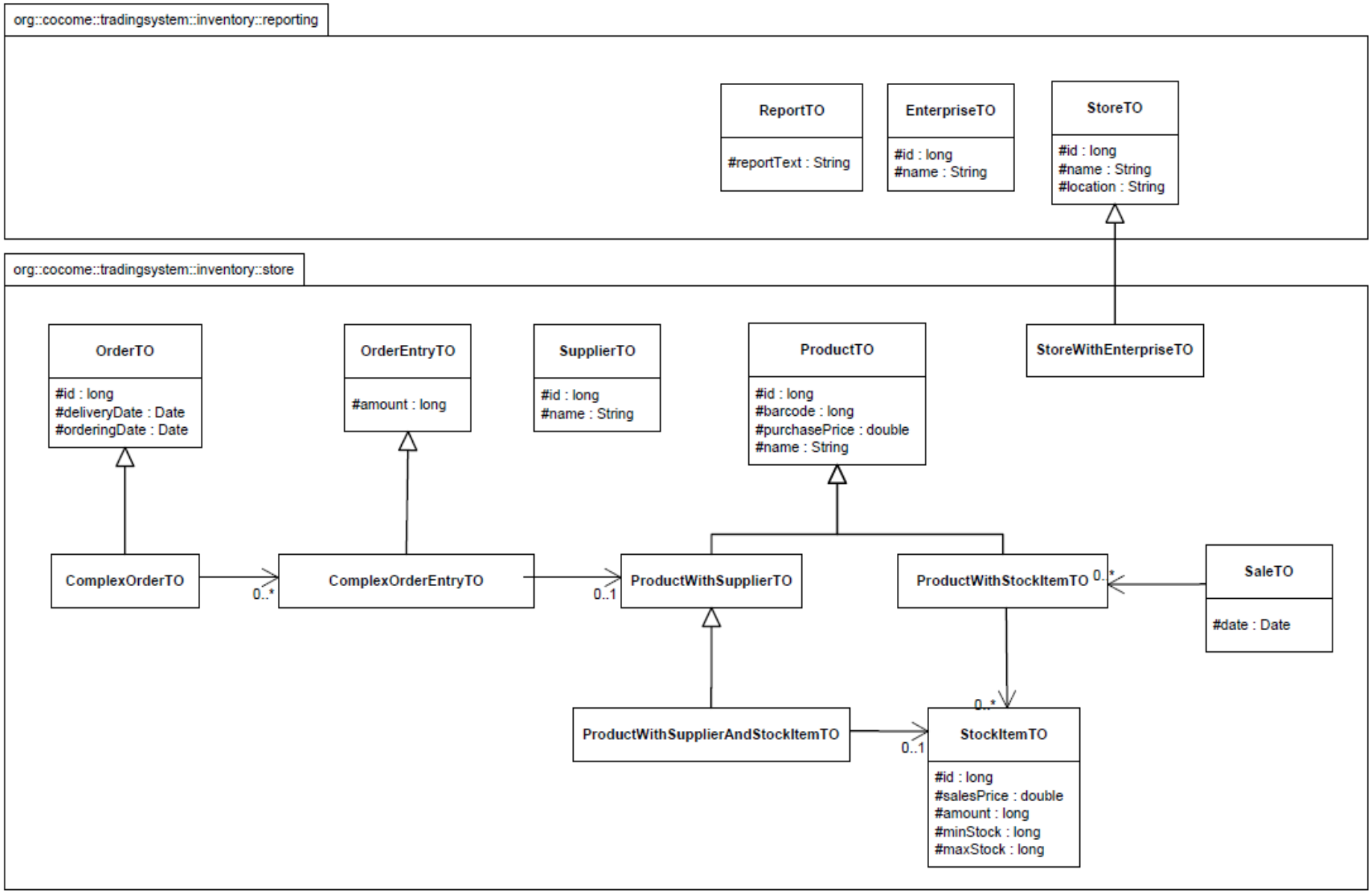
Alkalmazási réteg

- Az alkalmazási (application) réteg három komponensből épül fel:
  - *Reporting,*
  - *Store,*
  - *ProductDispatcher*.
- A Reporting komponens valósítja meg a ReportinfIf interfészt.
- A Store komponens valósítja meg a CashDeskConnectorIf és a StoreIf interfészeket.
- A Store komponens elvárt interfésze a SaleRegisteredEvent és a ProductDispatcherIf.
- ProductDispatcherIf definiál az Enterprise Server számára egy metódust, amely egy másik áruházban keres termékeket.
- A kommunikáció a Data és az Aplication között azáltal valósul meg, hogy a Data perzisztensz objektumok referenciáit adja át az Application komponensnek.
- Az Application réteg Transfer Objects (TO) felhasználásával ad át információt a GUI és a CashDeskLine komponenseknek.

Az Inventory komponens alkalmazási rétegének belső struktúrája



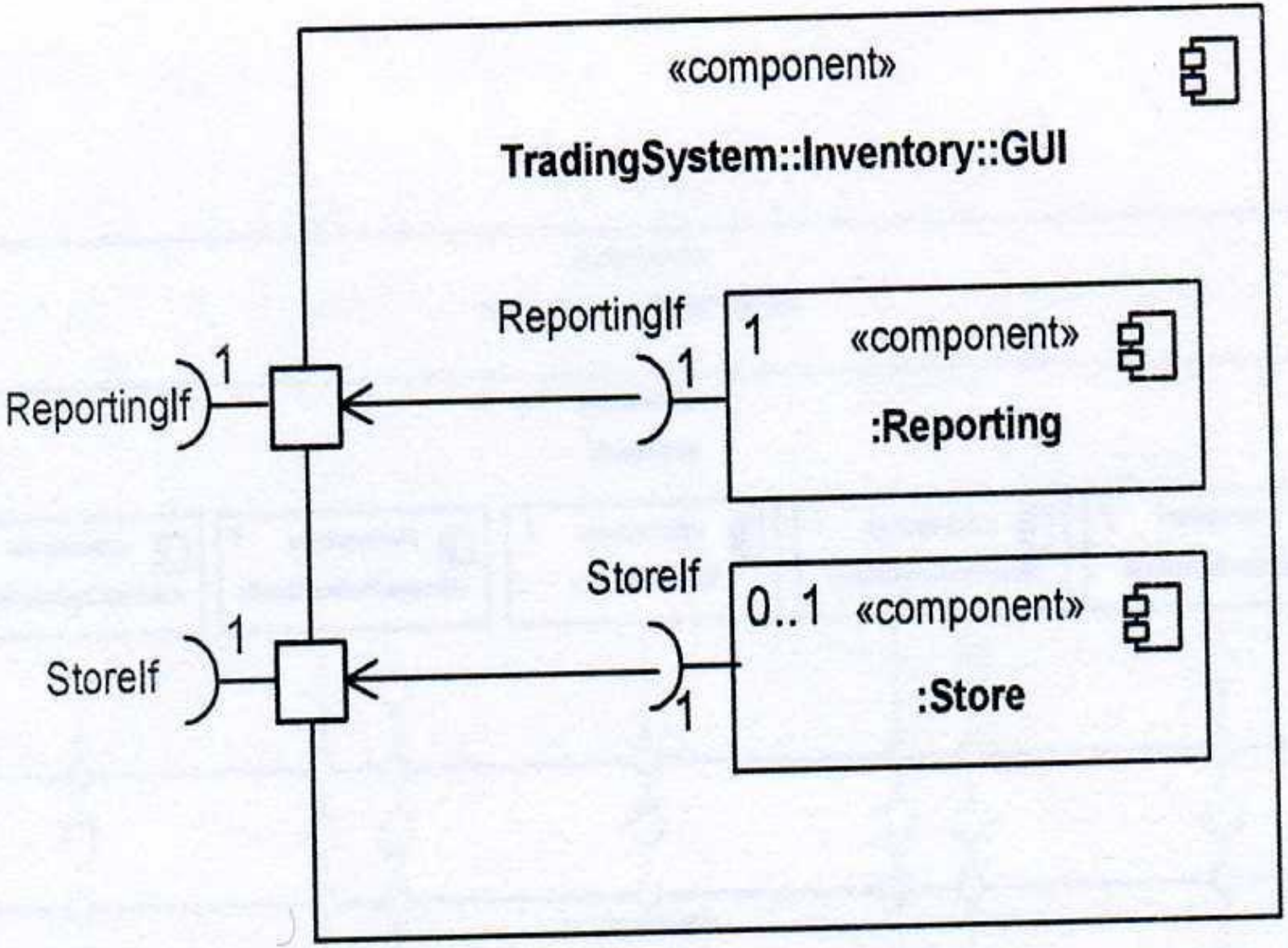
Adatcserét megvalósító Transfer Objects az alkalmazási és a GUI



GUI réteg belső struktúrája

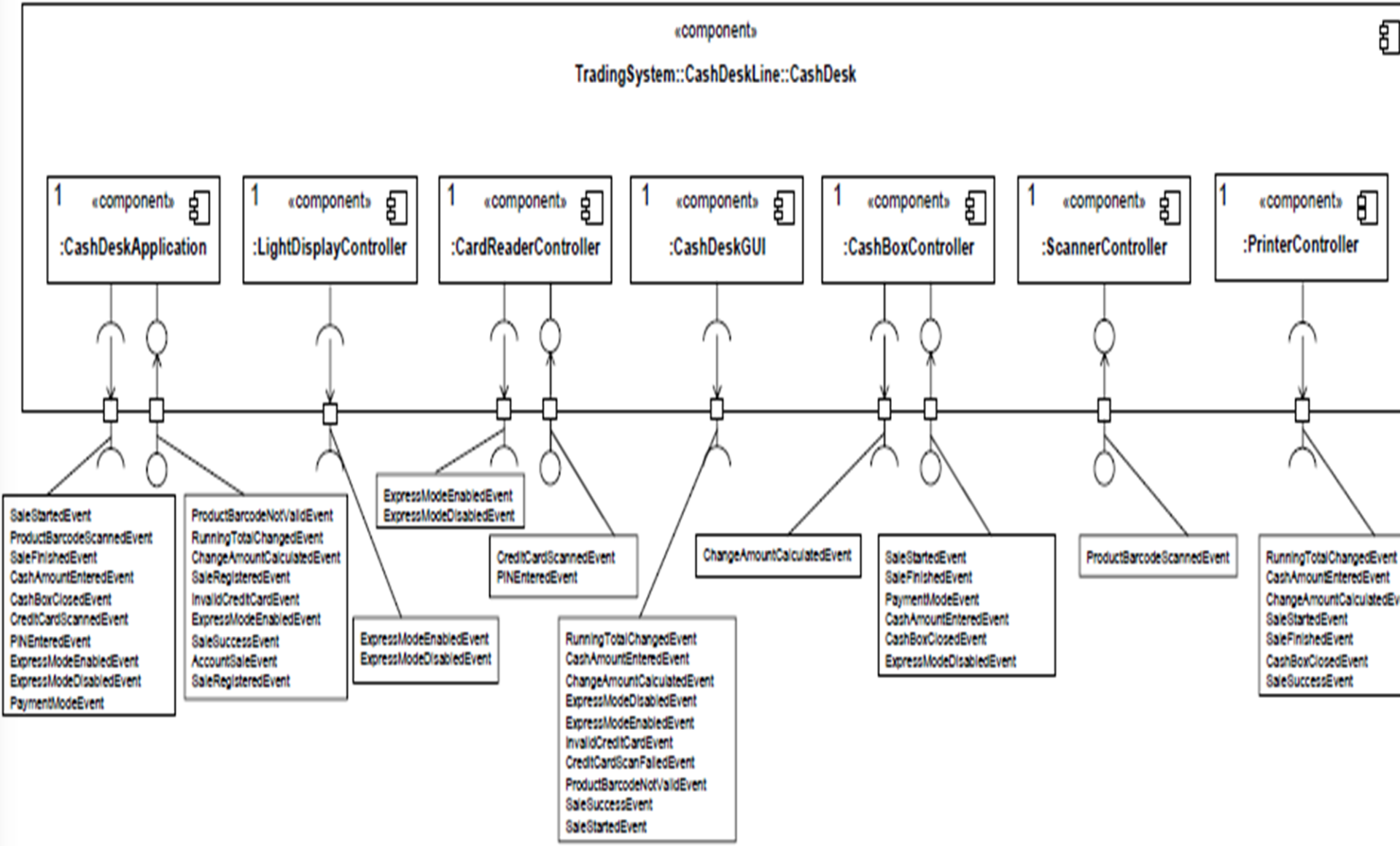
- Az Inventory komponens GUI rétegének belső struktúráját a következő ábra mutatja be.
- A GUI rétegnek két alkomponense van
  - Reporting
    - Ez a komponens valósítja meg a különböző jelentések (reports) vizualizálását, amelyekhez az adatokat a ReportingIf interfészen keresztül kapja meg.
  - Store
    - Ez a komponens a Store Manager számára nyújt felhasználói interfészt, amelyen keresztül az menedzseli az „ordering products” vagy „changing the sale prices” , stb. taszkokat.

Az Inventory komponens GUI rétegének belső struktúrája



Pénztársor (Cash Desk Line) komponens strukturális nézete

- Ez a komponens a rendszer beágyazott részét képezi.
- Feladata az összes pénztár menedzselése beleértve a hardverüket, a pénztárak közötti együttműködést és az egyes pénztárak és a hozzájuk csatlakoztatott eszközök közötti kapcsolatokat is.
- A fő kommunikáció a az eseménycsatornákon keresztül elküldött események formájában valósul meg.
- A következő ábrán a CashDeskLine komponens belső struktúrájának áttekintő ábráját mutatjuk be.
- A CasDeskLine komponens tartalmazza:
  - CashDesk több példányát,
  - EventBus komponenst,
  - Coodinator komponenst.



of the currently scanned product using the barcode.

Connectorif

sales are sent through this interface to the Inventory.

EventBus komponens

- Ez a komponens menedzseli az EventChanel két példányát, amelyek a CasDesk minden példánya számára közösek:
  - *cashDeskChannel*
    - A CashDesk használja a hozzá csatolt eszközök vezérlői közötti kommunikációra
      - *CashDeskApplication, LightDisplayController, stb.*
      - Minden vezérlőhöz egy-egy hardvereszköz csatlakozik
      - Ezek együtt képezik a hidat a hardver és a köztesréteg (middleware) között.
  - *extCommChannel*
    - *A CashDeskApplication komponens ezen keresztül írja ki az Inventory komponensbe az információt a befejezett eladásokról.*
    - *A Coordinátor , amely az expressz pénztári szolgáltatásokat menedzseli és a CashDesk komponensek is használják ezt a csatornát.*

A CasDesk komponens részletezése

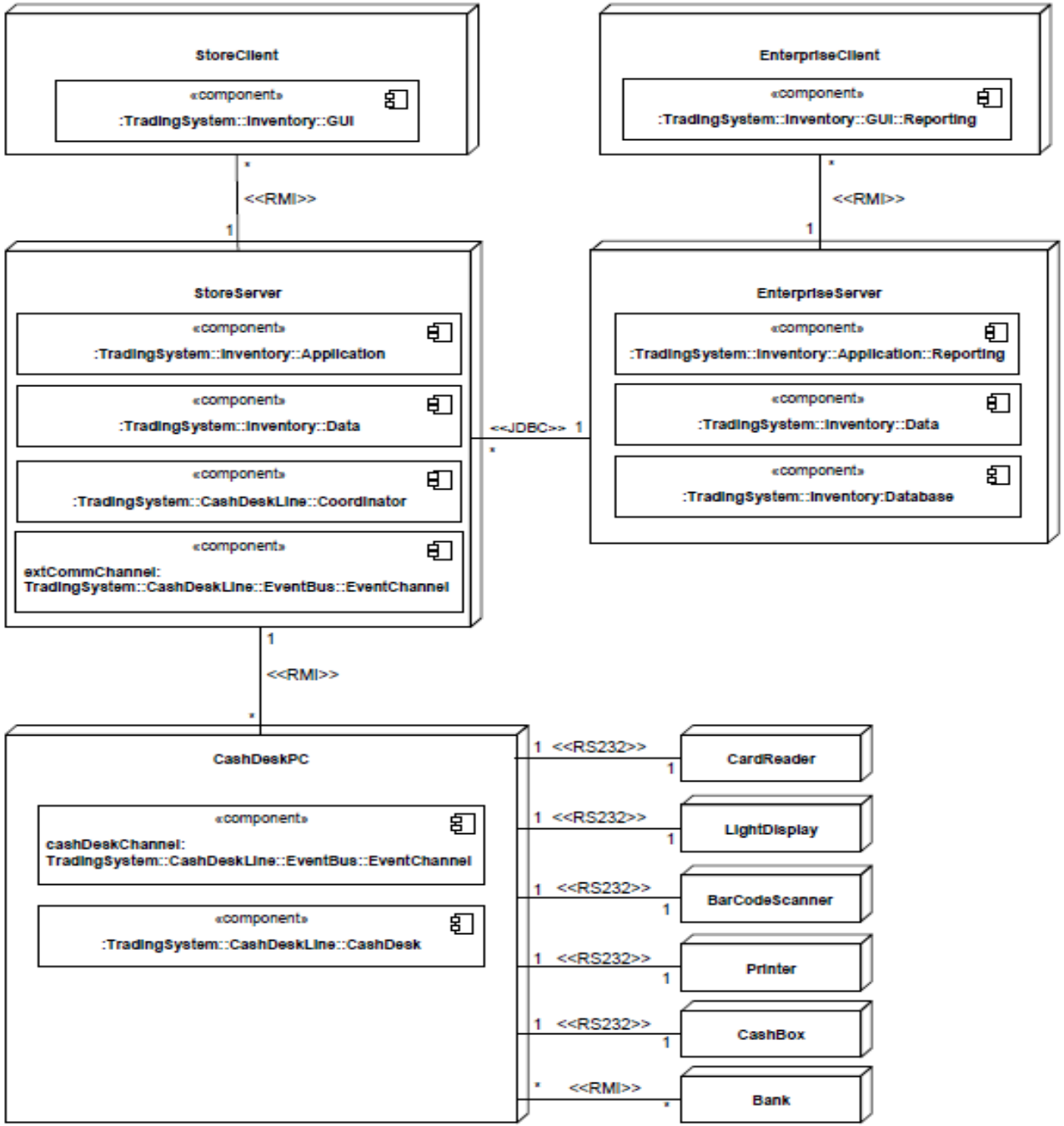
- Amint az előző ábrán láthattuk a CashDesk komponensek üzeneteket is kezelnek.
- Az ábrán félkörök jelölik azokat az üzeneteket, amelyeket a komponens kezelhet, a teljes körök pedig azokat az üzeneteket jelölik, amelyeket a komponens más komponenseknek elküld.
  - Például a CardReaderController vezérlő
    - kezeli az *ExpressModeEnabledEvent* eseményt és
    - elküldi a *CreditCardScannedEvent* és a *PINEnteredEvent* eseményeket.

A kereskedelmi rendszer telepítési nézete

- Azzal a kérdéssel foglakozik, hogy a kereskedelmi rendszer milyen eszközök segítségével valósítható meg.
  - Minden pénztárhoz egy saját PC tartozik, amelyhez további eszközök csatlakoztathatók
    - Vonalkód leolvasó vagy kártyaolvasó
      - A fenti eszközök vezérlői a PC-n futnak, a PC velük egy esemény csatornán keresztül tartja a kapcsolatot.
  - Minden áruházhoz tartozik egy szerver (Store Server)
    - Minden pénztári PC ehhez csatlakozik
    - Az áruházi szerver komponens további négy komponensből áll: Coordinator, extCommChannel, Application és Data.

Telepítési nézet

Az egyes komponensek fizikai (pl. hardver) elhelyezkedése.



Viselkedési modell

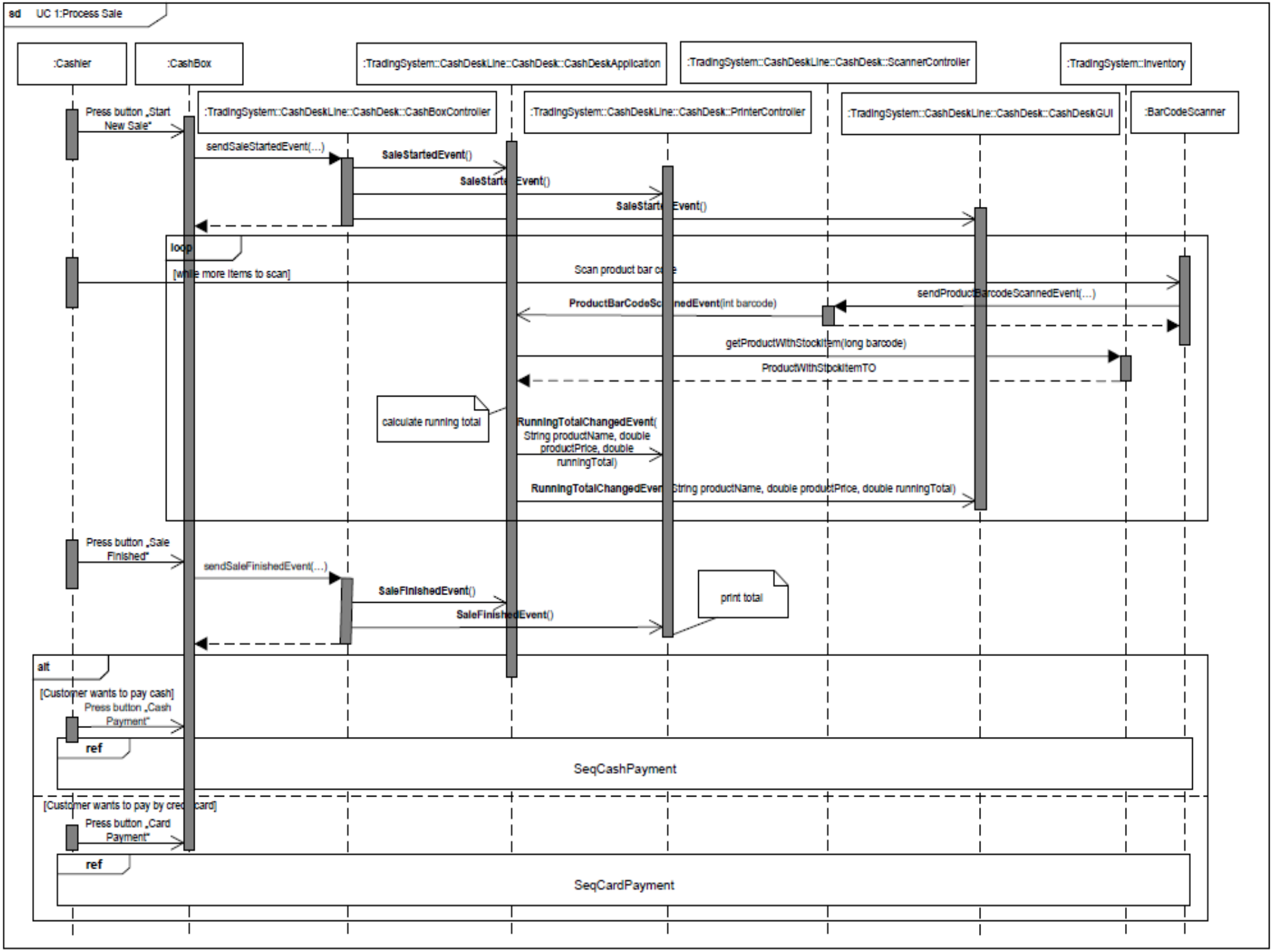
- Az egyes használati esetek formálisabb leírása valamely modellező eszköz segítségével
- Jelen esetben UML 2.0 szekvencia diagramok használatával

A kereskedelmi rendszer viselkedési nézete

- Ebben az esetben a rendszer viselkedési nézetét a használati esetek szekvencia diagramjaival adjuk meg.
- A szekvencia diagramokon a következő jelöléseket használjuk:
  - Kitöltött fejű nyíl jelöli a szinkron metódushívásokat,
  - Nem kitöltött fejű nyíllal jelöljük az aszinkron módon meghívott metódusokat.
- A szereplők és a komponensek közötti együttműködést írjuk le a szekvencia diagramokkal.

Az UC 1 használati eset viselkedési nézete

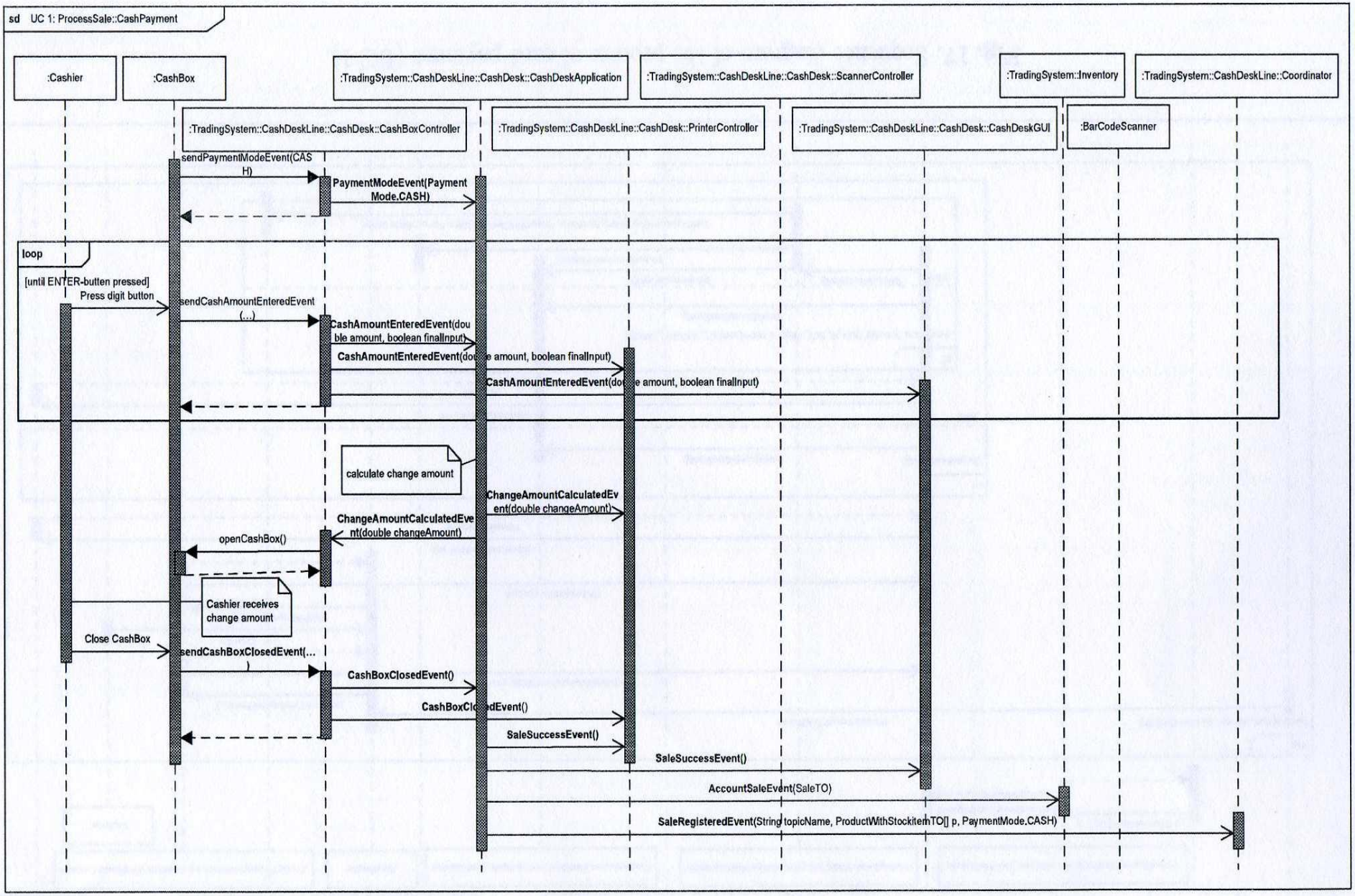
- Az árusítás folyamata használati eset (UC 1) szekvencia diagramját a következő három ábra tartalmazza.
  - A fő folyamat (sd UC 1: Process Sale) szekvencia diagramján szerepelnek
    - a pénztáros valamint további
    - szoftverkomponensek
      - CashBox
      - CashBoxController
      - CashDeskApplication
      - PrinterController
      - ScannerController
      - CashDeskGUI
      - Inventory
      - BarCodeScanner


sd UC 1:Process Sale
:Cashier
:CashBox
:TradingSystem::CashDeskLine::CashDesk::CashDeskApplication
:TradingSystem::CashDeskLine::CashDesk::ScannerController
:TradingSystem::Inventory
:TradingSystem::CashDeskLine::CashDesk::CashBoxController
:TradingSystem::CashDeskLine::CashDesk::PrinterController
:TradingSystem::CashDeskLine::CashDesk::CashDeskGUI
:BarCodeScanner
Press button „Start New Sale"
sendSaleStartedEvent(...)
SaleStartedEvent()
SaleStartedEvent()
SaleStartedEvent()
loop
[while more items to scan]
Scan product bar code
sendProductBarcodeScannedEvent(...)
ProductBarCodeScannedEvent(int barcode)
getProductWithStockItem(long barcode)
ProductWithStockItemTO
calculate running total
RunningTotalChangedEvent(String productName, double productPrice, double runningTotal)
RunningTotalChangedEvent(String productName, double productPrice, double runningTotal)
Press button „Sale Finished"
sendSaleFinishedEvent(...)
SaleFinishedEvent()
SaleFinishedEvent()
print total
alt
[Customer wants to pay cash]
Press button „Cash Payment"
ref
SeqCashPayment
[Customer wants to pay by credit card]
Press button „Card Payment"
ref
SeqCardPayment

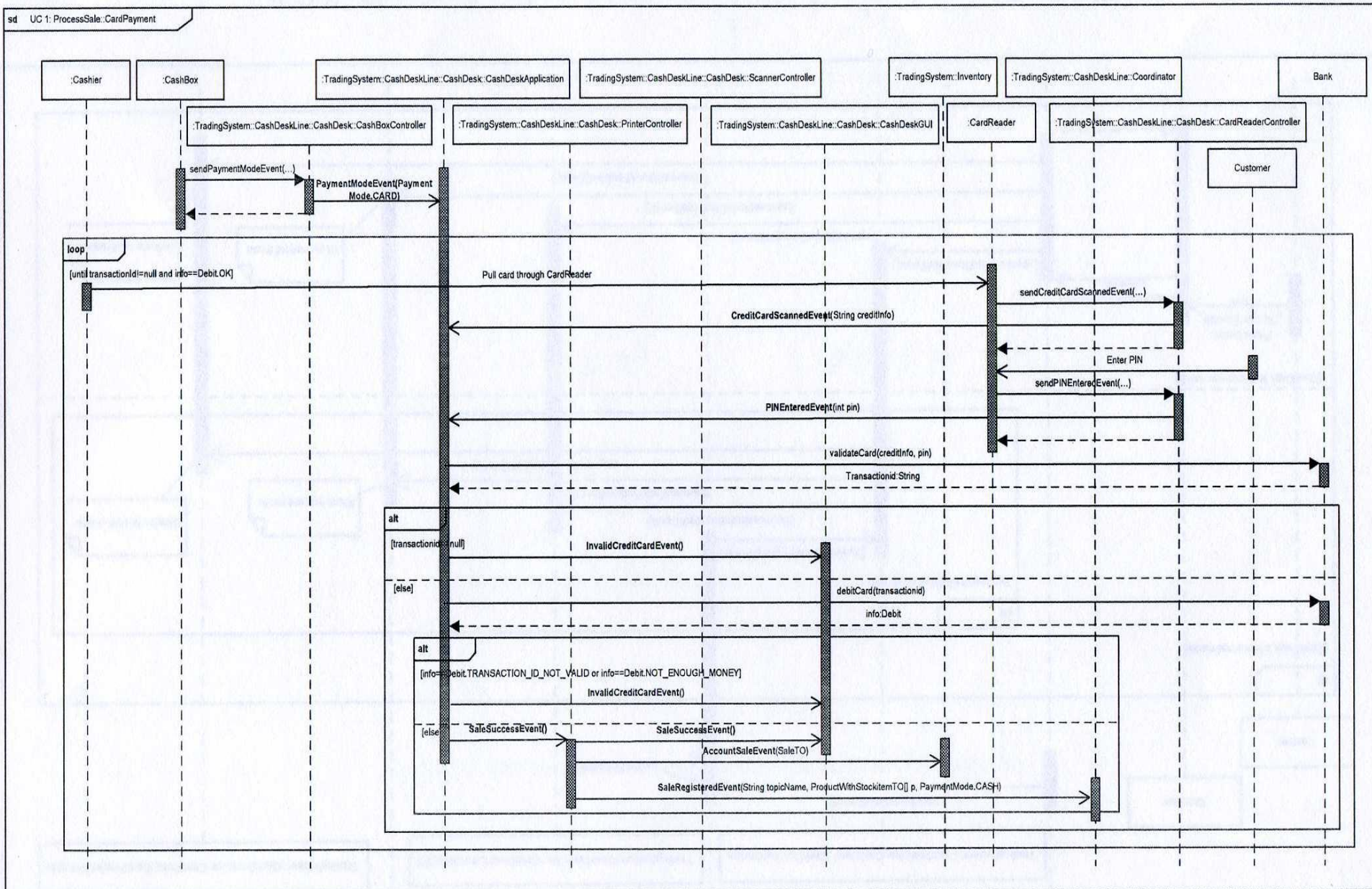
Fő folyamat: *sd UC 1:Process Sale*

- A fő folyamat tevékenységei:
  - A pénztáros megnyomja az „új vásárlás indul” gombot, amelynek hatására
    - egy új vásárlás inicializálása megtörténik valamint
    - a printer az új vásárlás fejlécét kinyomtatja a számlára.
  - A pénztáros a vonalkód leolvasóval leolvassa az egyes tételek kódját, amely alapján az
    - Inventory komponens megadja a CashDeskApplication komponens számára az áru megfelelő adatait, amely alapján felkerül a számlára az újabb tétel.
    - Ez ismétlődik amíg a vásárolt tételek mindegyikét feldolgozza a pénztáros és megnyomja a „vásárlás vége” gombot.
  - A vásárló kétféle módon fizethet:
    - Készpénzzel
      - Sd UC 1: ProcessSale:CashPayment
    - Bankkártyával
      - Sd UC 1: ProcessCardPayment.

UC 1 fizetés készpénzzel

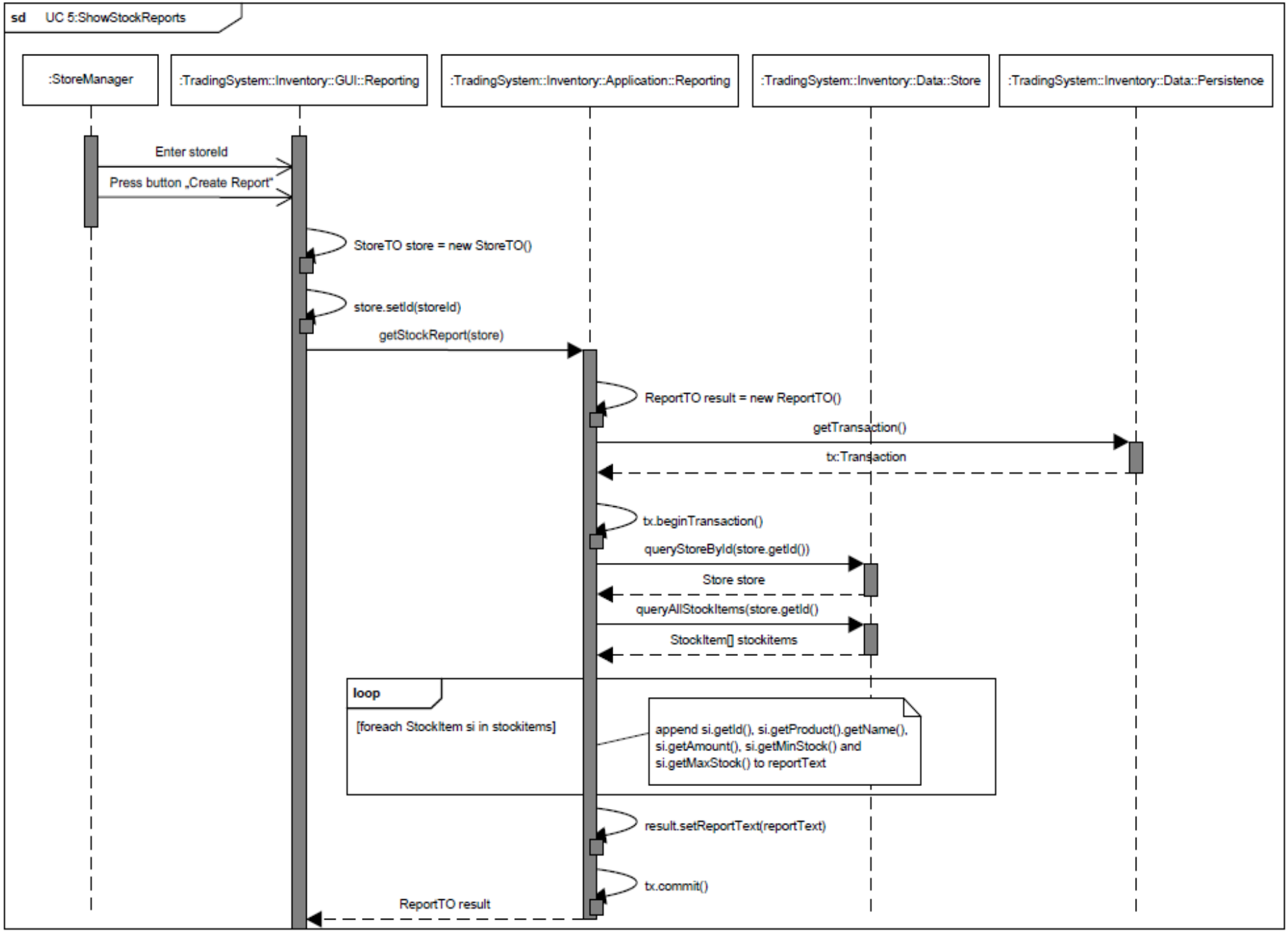


UC 1 fizetés kártyával



Az UC 5 használati eset viselkedési nézete

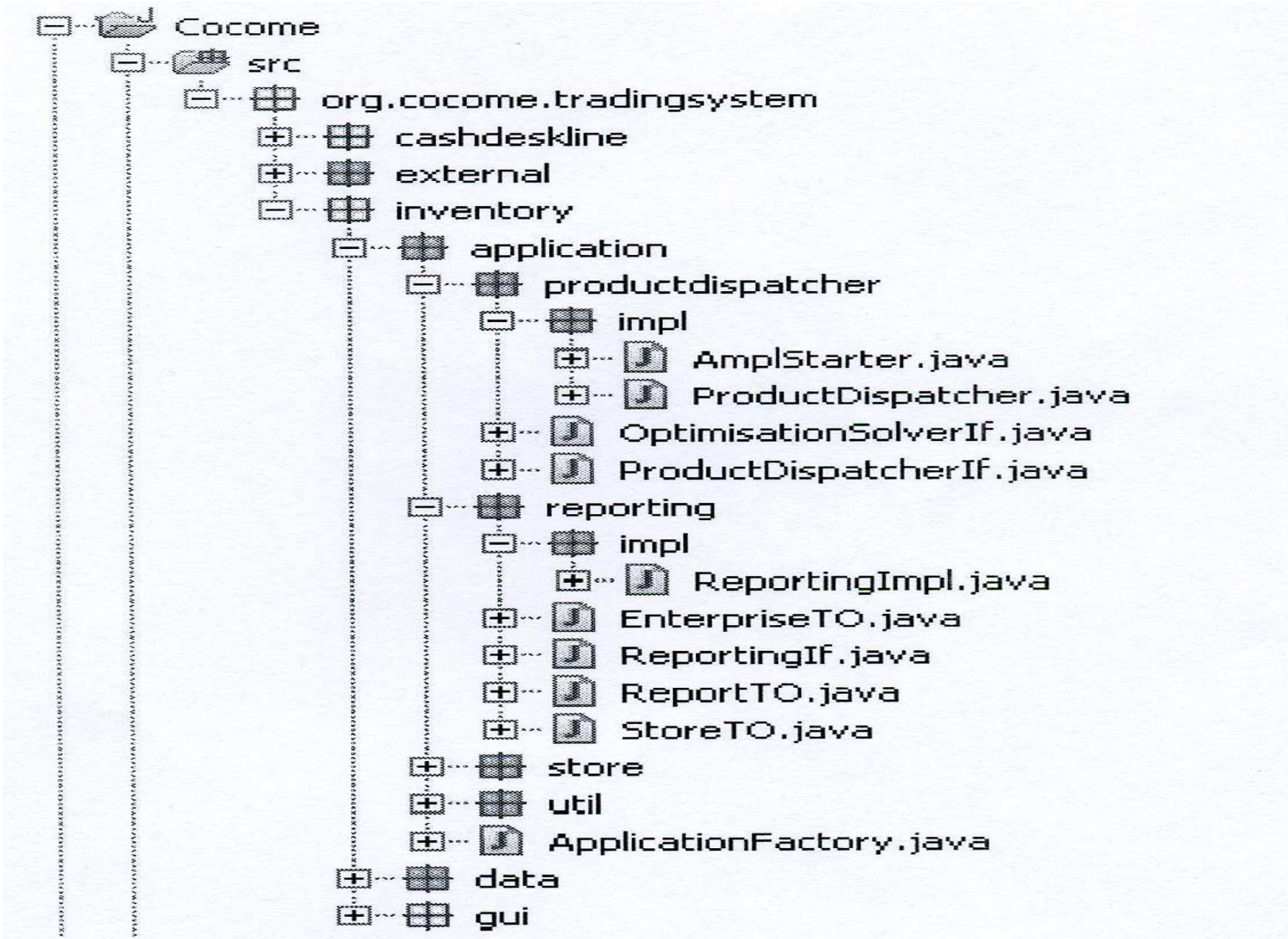
- UC 5 Show Stock reports
- Az áruház menedzsere információt kérhet arról, hogy mely árucikkek vannak kifogyóban.
  - A lista generálására a GUI komponens Reporting szolgáltatását használhatja megadva az áruház azonosítóját és megnyomva a CreateReport gombot.
  - A GUI Reporting komponense getStockReport() szolgáltatása ezután a Data::Store komponensétől megszerzi a kívánt információt és generálja a listát, ahogy az a következő ábrán látható.
  - Az eredmény egy *ReportTO* objektum formájában testesül meg, amelyet megküld a GUI::Reporting komponensnek, ahol az áruház menedzsere megtekintheti az eredményt.


sd UC 5:ShowStockReports
:StoreManager
:TradingSystem::Inventory::GUI::Reporting
:TradingSystem::Inventory::Application::Reporting
:TradingSystem::Inventory::Data::Store
:TradingSystem::Inventory::Data::Persistence
Enter storeId
Press button „Create Report"
StoreTO store = new StoreTO()
store.setId(storeId)
getStockReport(store)
ReportTO result = new ReportTO()
getTransaction()
tx:Transaction
tx.beginTransaction()
queryStoreById(store.getId())
Store store
queryAllStockItems(store.getId()
StockItem[] stockitems
loop
[foreach StockItem si in stockitems]
append si.getId(), si.getProduct().getName(), si.getAmount(), si.getMinStock() and si.getMaxStock() to reportText
result.setReportText(reportText)
tx.commit()
ReportTO result

Implementáció

- A kereskedelmi rendszert Java nyelven valósítottuk meg.
- A komponenseknek egy-egy Java csomag (package) felel meg.
- A csomagból kifelé csak a benne foglalt komponens által definiált külső felületek (interface) látszanak.
- Az interfészeket megvalósító osztályok az „impl” alcsomagban találhatók.

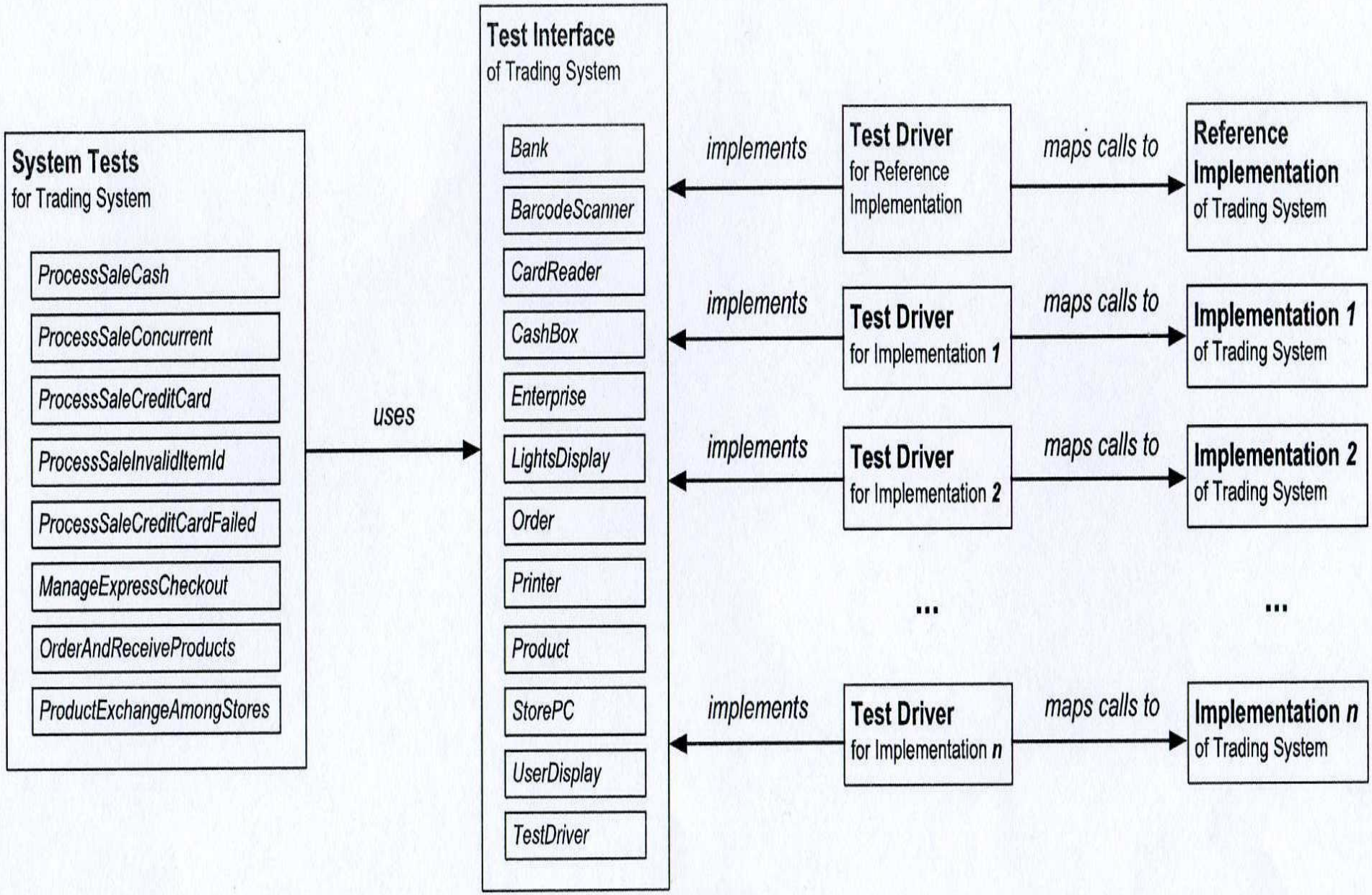
Implementáció



A rendszer tesztelése

- A tesztelési keretrendszer kettős feladatot tölt be:
  - a rendszer bizonyos aspektusairól kapjunk részletesebb információt,
  - a különböző részeket modellező csapatok lehetőleg automatikusan tesztelhessék az implementációjukat.
- A fenti célok elérésére a JUnit tesztelési keretrendszert használták a projektben a tesztelési forgatókönyvek elkészítése közben is.
- Az elkészült tesztelési keretrendszer több réteget tartalmaz,a hogy azt a következő ábra mutatja.
  - A teszteléshez közös tesztelési interfészt használnak, amelyet az úgynevezett tesztvezérlők (test drivers) valósítanak meg.
  - A teszt forgatókönyveket ezeken az interfészeken keresztül valósítjuk meg.
  - Ezáltal lehetővé válik, hogy a rendszer újabb megvalósításai esetén a tesztelést újra elvégezhessük csak a megfelelő tesztvezérlőt kell elkészíteni.

A tesztelési keretrendszer architektúrája



Tesztelési forgatókönyvek

- Egy tesztelési forgatókönyv (test Scenarios) esetünkben a kereskedelmi rendszer használati eset leírásán alapul.
- Kétféle módon adhatók meg a tesztelési forgatókönyvek
  - formálisan Java osztályok formájában
    - a tesztelés automatikus a JUnit felhasználásával,
  - informálisan leírt módon
    - Ebben az esetben a tesztelés manuálisan történik
      - use case 5 jó példa erre.
- A következő táblázatokban megadjuk a use case azonosítóját, a hozzá tartozó teszteset azonosítóját, a teszteset típusát (Type) és a rövid leírásukat is.
- A formalizált teszteset esetében a JUnit a forráskód alapján megadja a teszt eredményét, informális esetben a „pass” vagy „fail” adjuk vissza a tesztelés eredményét.

Néhány konkrétabb teszt eset

Használati eset	Teszt eset	Típus	Leírás
1	Árusítás készpénzzel	Java	Néhány árú vársárlása készpénzes fizetéssel. Nincs hiba
1	Konkurens árusítás	Java	Vásárlás egyszerre több pénztárnál
1	Árusítás bankkártyával	Java	Néhány árú vásárlása fizetés bankkártyával. Nincs hiba
1	Árusítás. Hibás termékazonosító	Java	Hibás termékazonosító kerül beolvasásra. Bevitel kézzel történik
5	Leltáradatok listázása	informális	A rendszernek lehetőséget kell biztosítania a Raktárba fellelhető árukról való lista készítésére. Test: Ellenőrizzük hogy a rendszer lehetővé teszi ezt. **PASS:** Előállítás lehetséges **FAIL:** Előállítás NEM lehetséges

Use Case	Test Case Id	Type	Description
1	ProcessSaleCash	Java	Purchase of some goods and cash payment, no exceptions.
1	ProcessSale Concurrent	Java	Concurrent purchase of some goods at n different cash desks, no exceptions.
1	ProcessSale CreditCard	Java	Purchase of some goods and credit card payment, no exceptions.
1	ProcessSale InvalidItemId	Java	Invalid item id read, manual entry of item id. (Exception in step 3 of use case 1)
1	ProcessSale CreditCardFailed	Java	Wrong PIN entry for credit card; card validation fails. (Exception in step 5.2 of use case 1)

3	ShowProducts ForOrdering	infor-mal	Store shall provide functionality to generate a report about products which are low on stock. **Test:** Check if System offers functionality for reporting products low on stock. **PASS:** Generation of report with products low on stock possible **FAIL:** Generation of report with products low on stock NOT possible
3, 4	OrderAnd ReceiveProducts	Java	Products low on stock will be ordered and correct delivery will be recorded in the inventory.

Use Case	Test Case Id	Type	Description
5	ShowStockReports	informal	System shall provide functionality to present a report including all available stock items in store or a report of cumulated available product items of a specified enterprise. **Test:** Check if System offers functionality for generation of stock reports **PASS:** Generation of stock reports possible **FAIL:** Generation of stock reports NOT possible
6	ShowDeliveryReports	informal	System shall provide functionality to present a report showing mean time to delivery for each supplier of a specific enterprise. **Test:** Check if System offers functionality for generation of a delivery report **PASS:** Generation of delivery report possible **FAIL:** Generation of delivery report NOT possible

7	ChangePrice	informal	System shall provide functionality to change sales price of a product. **Test:** Check if System offers functionality for change sales price of a product **PASS:** Change of sales price for product item possible **FAIL:** Change of sales price for product item NOT possible
8	ProductExchange AmongStores	Java	After a sale which leads to a product being low on stock of the store, product exchange between stores should take place.

Az előadáshoz ajánlott irodalom

1. Larman, C.: Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development (3rd Edition). Prentice Hall PTR (2004)
2. OMG, Object Management Group: UML Profile for Schedulability, Performance and Time. http://www.omg.org/cgi-bin/doc?formal/2005-01-02 (2005)
3. SUN Microsystems: The Java Database Connectivity (JDBC). (http://java.sun.com/javase/technologies/database/index.jsp)
4. JBoss (Red Hat Middleware): Hibernate. (http://www.hibernate.org)
5. SUN Microsystems: Java Persistence API. (http://java.sun.com/javaee/technologies/persistence.jsp)
6. SUN Microsystems: Java Message Service. (http://java.sun.com/products/jms/)
7. Apache: The Apache Ant Project. (http://ant.apache.org)
8. JUnit: JUnit. (http://www.junit.org)